

Analyses: Classification with KNN

Description

K-Nearest Neighbor classification of the feature vectors per character sequence (loaded from `~/results/features_combined.csv`). The results are stored in `~/results/classifier/`. KNN is strictly speaking not a machine “learning” algorithm. Given a data point in the test set, it simply takes the k-nearest neighbors (in the “training” set) of this data point, and assigns the majority label of these to it. See figure below.

In the context of the current analyses, there are four relevant dimensions (instead of two as in the example below), namely, the TTR, unigram entropy, entropy rate, and repetition rate for the respective sequences.

```
knitr::include_graphics("~/Github/PaleoSigns/figures_external/knn.jpg")
```

Session Info

Give the session info (reduced).

```
## [1] "R version 4.4.0 (2024-04-24)"  
## [1] "x86_64-pc-linux-gnu"
```

Load Packages

If the libraries are not installed yet, you need to install them using, for example, the command: `install.packages("ggplot2")`.

```
library(dplyr)  
library(class)  
library(gmodels)  
library(caret)
```

Give the package versions.

```
##      caret  lattice  ggplot2  gmodels    class    dplyr  
## "6.0-94" "0.22-5"  "3.5.1" "2.19.1" "7.3-22"  "1.1.4"
```

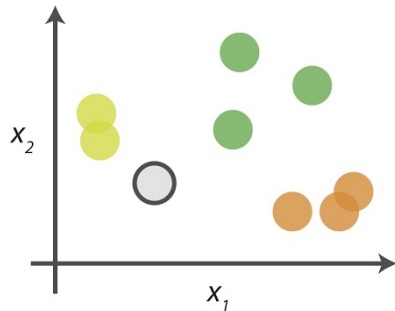
Load Data

Load data table with estimated feature values per sequence.

```
# load feature value estimations  
estimations.df <- read.csv("~/Github/PaleoSigns/results/features/features_combined.csv")  
head(estimations.df)
```

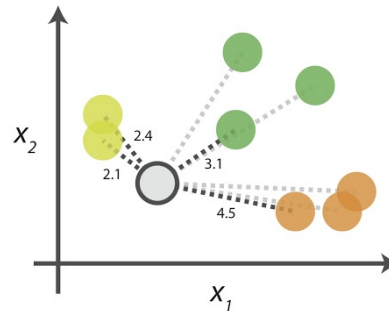
kNN Algorithm

0. Look at the data



Say you want to classify the grey point into a class. Here, there are three potential classes - lime green, green and orange.

1. Calculate distances



Start by calculating the distances between the grey point and all other points.

2. Find neighbours

Point Distance			
	→		2.1 → 1st NN
	→		2.4 → 2nd NN
	→		3.1 → 3rd NN
	→		4.5 → 4th NN

Next, find the nearest neighbours by ranking points by increasing distance. The nearest neighbours (NNs) of the grey point are the ones closest in dataspace.

3. Vote on labels

Class	# of votes	
	2	Class wins the vote! Point is therefore predicted to be of class .
	1	
	1	

Vote on the predicted class labels based on the classes of the k nearest neighbours. Here, the labels were predicted based on the k=3 nearest neighbours.

Figure 1: Workings of the KNN algorithm, taken from <https://ourcodingclub.github.io/tutorials/machine-learning/>.

```
##      id      subcorpus      sequence num.tokens num.types huni.chars
## 1 bst0001 Aurignacien vvvvvvvvvvvvvvvvvv      18         1 0.0000000
## 2 gkl0001 Aurignacien   ||| /////_V      12         5 1.8962406
## 3 gkl0002 Aurignacien                vvvvv      5         1 0.0000000
## 4 gkl0003 Aurignacien          vvvv vvvv      9         2 0.5032583
## 5 gkl0004 Aurignacien      vvvvvvvvvvvvvvvv     14         1 0.0000000
## 6 gkl0005 Aurignacien                |||      3         1 0.0000000
##      hrate.chars  ttr.chars  rm.chars
## 1      0.6171397 0.05555556 1.0000000
## 2      1.5295563 0.41666667 0.6363636
## 3      0.5711903 0.20000000 1.0000000
## 4      0.9060047 0.22222222 0.7500000
## 5      0.6379550 0.07142857 1.0000000
## 6      0.4308271 0.33333333 1.0000000
```

Select subcorpora for binary classification.

```
# choose subcorpora to compare: "Aurignacien", "TeDDi", "Cuneiform (Uruk V)",
# "Cuneiform (Uruk IV)", "Cuneiform (Uruk III)".
subcorpus1 <- "Aurignacien"
subcorpus2 <- "Cuneiform (Uruk III)"
selected <- c(subcorpus1, subcorpus2)
estimations.subset <- estimations.df[estimations.df$subcorpus %in% selected, ]
nrow(estimations.subset)
```

```
## [1] 1066
```

```
# downsample to have the same number of sequences across different combinations of corpora
# comment this out to use all the sequences (unbalanced across combinations of corpora)
# set.seed(02152018)
# estimations.subset <- sample_n(estimations.subset, 300)

# exclude sequences under a certain length
#estimations.subset <- estimations.subset[estimations.subset$num.tokens > 10, ]
```

Remove NAs (whole row)

```
estimations.subset <- na.omit(estimations.subset)
```

Normalization

Center and scale the data.

```
# first four columns are meta information, the others are the feature values
# be aware that this needs to be changed in case of other table format
estimations.scaled <- cbind(estimations.subset[, c("id", "subcorpus", "sequence",
                                                    "num.tokens", "num.types")],
                             scale(estimations.subset[, c("huni.chars", "hrate.chars",
                                                            "ttr.chars", "rm.chars")]))
```

Training and Test Sets

```

# Generating seed
set.seed(09012020)
# Randomly generating our training and test samples with a respective ratio of 2/3 and 1/3
datasample <- sample(2, nrow(estimations.scaled), replace = TRUE, prob = c(0.67, 0.33))

# Generate training set (select rows randomly by using datasample vector)
train <- estimations.scaled[datasample == 1, 1:ncol(estimations.scaled)]
# Generate test set
test <- estimations.scaled[datasample == 2, 1:ncol(estimations.scaled)]
# number of data points in test set
test.n <- nrow(test)

```

Building KNN Classifier

Initialize data frame for storing results.

```

knn.results <- data.frame(k = numeric(0), baseline = numeric(0),
                          accuracy = numeric(0), accuracy.pvalue = numeric(0),
                          precision = numeric(0), recall = numeric(0),
                          f1 = numeric(0), test.n = numeric(0))

```

Run KNN classification.

```

# choose maximum number of neighbors n
n = 10
# run a loop over different numbers of neighbors up to n
for (k in 1:n){
  # knn estimation of labels
  predictions.knn <- knn(train = train[, c("huni.chars", "hrate.chars",
                                           "ttr.chars", "rm.chars")],
                        test = test[, c("huni.chars", "hrate.chars",
                                         "ttr.chars", "rm.chars")],
                        cl = train[, "subcorpus"], k = k)

  # model evaluation
  # creating a dataframe from known (true) test labels
  test.labels <- test[, "subcorpus"]
  # combining predicted and known classes
  class.comparison <- data.frame(predictions.knn, test.labels)
  # giving appropriate column names
  names(class.comparison) <- c("predicted", "observed")
  # inspecting our results table
  head(class.comparison)
  # get confusion matrix
  cm <- confusionMatrix(data = class.comparison$predicted,
                       reference = as.factor(class.comparison$observed))

  # print(cm)
  # get performance metrics from the output list of confusionMatrix()
  baseline <- cm$overall['AccuracyNull']
  accuracy <- cm$overall['Accuracy']
  accuracy.pvalue <- cm$overall['AccuracyPValue']
  f1 <- cm[["byClass"]][["F1"]]
  recall <- cm[["byClass"]][["Recall"]]
}

```

```

precision <- cm[["byClass"]]["Precision"]

# unname and round resulting values (except for p-values)
baseline <- round(unname(baseline), 2)
accuracy <- round(unname(accuracy), 2)
f1 <- round(unname(f1), 2)
recall <- round(unname(recall), 2)
precision <- round(unname(precision), 2)

# prepare data frame with results
local.results <- data.frame(k, baseline, accuracy, accuracy.pvalue,
                           precision, recall, f1, test.n,
                           row.names = NULL)

# print(local.results.rounded)
knn.results <- rbind(knn.results, local.results)
# give predictions alongside the other information in data frame
knn.predictions.df <- cbind(test, predictions.knn)
# write df with predictions for post-hoc analyses
write.csv(knn.predictions.df, file =
          paste(c("~/Github/PaleoSigns/results/classifier/KNN/predictions/predictions_KNN_",
                  subcorpus1, "_",
                  subcorpus2, "_",
                  paste("k", k, sep = ""),
                  ".csv"), collapse = ""),
          row.names = F)
}
print(knn.results)

```

##	k	baseline	accuracy	accuracy.pvalue	precision	recall	f1	test.n
## 1	1	0.79	0.93	1.232907e-12	0.81	0.87	0.84	339
## 2	2	0.79	0.93	1.435439e-11	0.83	0.81	0.82	339
## 3	3	0.79	0.93	1.435439e-11	0.82	0.83	0.82	339
## 4	4	0.79	0.92	1.412783e-10	0.79	0.83	0.81	339
## 5	5	0.79	0.92	1.412783e-10	0.79	0.84	0.81	339
## 6	6	0.79	0.92	4.595375e-11	0.81	0.83	0.82	339
## 7	7	0.79	0.92	4.595375e-11	0.80	0.84	0.82	339
## 8	8	0.79	0.93	1.232907e-12	0.82	0.86	0.84	339
## 9	9	0.79	0.93	4.299527e-12	0.82	0.84	0.83	339
## 10	10	0.79	0.93	1.435439e-11	0.81	0.84	0.83	339

Write results to file.

```

# write results
write.csv(knn.results,
          file = paste(c("~/Github/PaleoSigns/results/classifier/KNN/performance/performance_KNN_",
                          subcorpus1, "_", subcorpus2, ".csv"), collapse = ""),
          row.names = F)

```

Package References

```

## Wickham H, François R, Henry L, Müller K, Vaughan D (2023). _dplyr_: A
## Grammar of Data Manipulation_. R package version 1.1.4,
## <https://CRAN.R-project.org/package=dplyr>.

```

```

## Warnes GR, Bolker B, Lumley T, Johnson RC, Jain N, Schwartz M, Rogers J
## (2024). _gmodels: Various R Programming Tools for Model Fitting_. R
## package version 2.19.1, <https://CRAN.R-project.org/package=gmodels>.

## Kuhn, Max (2008). "Building Predictive Models in R Using the caret
## Package." _Journal of Statistical Software_, *28*(5), 1-26.
## doi:10.18637/jss.v028.i05 <https://doi.org/10.18637/jss.v028.i05>,
## <https://www.jstatsoft.org/index.php/jss/article/view/v028i05>.

## Venables WN, Ripley BD (2002). _Modern Applied Statistics with S_,
## Fourth edition. Springer, New York. ISBN 0-387-95457-0,
## <https://www.stats.ox.ac.uk/pub/MASS4/>.

```